

Autenticação no SIGAA

v 1.0

Followw UnB · Guia técnico e plano de testes

SIGAA → requisição HTTP → sessão autenticada → token JWT → rotas protegidas

1 Objetivo

O Followw UnB utiliza o módulo de autenticação para conectar com segurança os alunos da UnB aos serviços restritos do SIGAA. O objetivo dos testes é garantir que as credenciais sejam validadas, os tokens JWT sejam gerados e verificados rigorosamente, e que falhas ou instabilidades do sistema legado sejam tratadas de forma elegante e resiliente sem expor dados sensíveis.

POST /api/auth/login → Validação Payload → Sessão SIGAA → Emissão JWT → Acesso Seguro

2 Fluxo de Implementação e Verificação

#	Etapa	Descrição
1	Recepção de Payload	Validação de formato, matrícula e senha não vazias via Pydantic
2	Comunicação SIGAA	Envio POST autenticado com ViewState e captura do cookie JSESSIONID
3	Validação de Resposta	Deteção de sucesso de login vs. mensagens de erro de credenciais
4	Geração do Token JWT	Criação de token seguro assinado contendo matrícula e expiração
5	Middleware de Sessão	Injeção de dependência FastAPI para proteção de rotas restritas
6	Tratamento de Falhas	Resiliência contra timeouts, instabilidades e manutenções do SIGAA

3 Testes Unitários — Validação e Tokens

Garantem que a lógica interna da aplicação funcione perfeitamente de forma isolada, sem necessidade de realizar chamadas de rede externas ou acessar o banco de dados.

```
# Teste de criação e validação de token JWT
token = create_access_token(data={"sub": "202012345"}, expires_delta=timedelta(hours=2))
payload = verify_access_token(token)
assert payload["sub"] == "202012345"
assert "exp" in payload
```

4 Mocking do SIGAA — Mantendo Testes Rápidos e Seguros

Como o SIGAA é um sistema em produção da Universidade de Brasília, os testes automatizados devem utilizar mocks (com bibliotecas como `respx` ou `unittest.mock`) para simular o comportamento do servidor sem sobrecarregar a infraestrutura da universidade nem arriscar bloqueios por IP.

```
@respx.mock
def test_sigaa_login_mock_sucesso(client):
    respx.post("https://sig.unb.br/sigaa/logar.do").respond(200, headers={"Set-Cookie": "JSESSIONID=123"})
    response = client.post("/api/auth/login", json={"matricula": "190012345", "senha": "senhaValida"})
    assert response.status_code == 200
    assert "access_token" in response.json()
```

5 Matriz de Casos de Teste (API de Autenticação)

Casos de teste essenciais cobrindo cenários felizes, entradas maliciosas e falhas externas:

Cenário de Teste	Payload / Entrada	Esperado	Descrição
Login Bem-Sucedido	Matrícula e senha válidas	200 OK	Retorna JWT e dados básicos
Credenciais Incorretas	Senha ou usuário errados	401 Unauth	Mensagem de erro amigável
Campos Obrigatórios	Matrícula ou senha vazias	422 Unproc	Erro de validação Pydantic
SIGAA Indisponível	Timeout ou erro 500 no SIGAA	504 / 503	Gateway timeout controlado
HTML Inesperado	Layout alterado no SIGAA	502 Bad Gtw	Falha limpa no parser JSF

6 Validação de Rotas Protegidas (Middleware)

Testes de integração para assegurar que nenhum endpoint restrito possa ser acessado sem um Bearer Token válido e não expirado:

Estado do Token	Cabeçalho HTTP	Resultado	Comportamento
Token Válido	Authorization: Bearer <token_ok>	200 OK	Acesso liberado aos recursos
Sem Cabeçalho	Nenhum header enviado	401 Unauth	Bloqueio imediato da requisição
Token Expirado	Authorization: Bearer <token_exp>	401 Unauth	Exige novo login do aluno
Assinatura Inválida	Authorization: Bearer <token_fake>	401 Unauth	Rejeição de token adulterado

7 Testes de Ponta a Ponta (E2E) e Logout

Os testes E2E validam o ciclo de vida completo da sessão: login, uso de rotas privadas e o logout. No logout, o aplicativo cliente descarta o token JWT e, caso haja uma blacklist no backend (Redis), o token atual é imediatamente invalidado antes da sua expiração natural.

```
def test_e2e_ciclo_autenticacao(client, mock_sigaa):  
    # 1. Realiza login e recebe o token  
    res_login = client.post("/api/auth/login", json={"matricula": "190012345", "senha": "correta"})  
    token = res_login.json()["access_token"]  
    # 2. Acessa endpoint protegido com sucesso  
    res_perfil = client.get("/api/user/perfil", headers={"Authorization": f"Bearer {token}"})  
    assert res_perfil.status_code == 200
```

8 Robustez, Boas Práticas e Ferramentas

A garantia de qualidade em software integrado a sistemas legados exige ferramentas modernas e cuidados com dados sensíveis:

- Pytest & pytest-asyncio: Execução automatizada e suporte a chamadas assíncronas no FastAPI.
- FastAPI TestClient (HTTPX): Simulação rápida e determinística de chamadas HTTP sem subir servidor real.
- Respx / Responses: Mock de conexões externas com o SIGAA para testes em CI/CD estáveis.
- Sanitização e Segurança: Garantir que senhas e dados acadêmicos nunca sejam expostos em logs de teste.
- Rate Limiting: Prevenção contra força bruta em rotas de login através de middlewares de limitação de taxa.

9 O Que Aprender com a Implementação

Construir uma suíte de testes para autenticação vai muito além de testar respostas 200 OK. Exige dominar o fluxo HTTP completo, o ciclo de vida de tokens criptográficos (JWT), o isolamento de dependências externas via Mocks e a resiliência no consumo de serviços legados.

Payload → Validação Pydantic → Mock SIGAA → Emissão JWT → Teste de Rota → Assertão